HOMEWORK 4

1.Implement a stack using queue data structure.

CODE:

```
#include <stdio.h>

#define MAX 100


int queue[MAX], front=-1, rear=-1;


void enqueue(int x){
   if(rear == MAX-1){
      printf("Queue is full\n");
      return;
   }


   if(front== -1){
      front =0;
   }
   rear++;
   queue[rear] = x;
}


int dequeue(){
```

```
    if(front ==-1 || front > rear){
       return -1;
    }

    int x = queue[front];
    front++;

    if(front > rear){
       front = rear = -1;
    }

    return x;

}

void push(int x){
    int i, n;

    if(front != -1)
      n = rear - front + 1;

    enqueue(x);
```

```
    for(i=0; i<n; i++){
        enqueue(dequeue());
    }
}

void pop(){
    int x;

    if(front==-1){
        printf("Stack is empty\n");
        return;
    }

    x = dequeue();

    printf("Popped element is %d\n",x);
}

void display(){
    int i;

    if(front == -1){
        printf("Stack is empty\n");
```

```
        return;
    }

    printf("Stack: ");

    for(i= front; i<=rear; i++){
        printf("%d ",queue[i]);
    }
    printf("\n");
}

int main(){
    int choice, value;

    printf("1.Push\n2.Pop\n3.Display\n4.Exit\n");

    do{
        printf("Enter your choice: ");
        scanf("%d",&choice);

        switch(choice){
            case 1:
              printf("Enter value: ");
```

```
            scanf("%d",&value);
            push(value);
            break;

          case 2:
            pop();
            break;

          case 3:
            display();
            break;

          case 4:
              printf("Exit\n");
              return 0;

       }
    }while(choice != 5);

    return 0;
}
```

OUTPUT:

1.Push

2.Pop

3.Display

4.Exit

Enter your choice: 1

Enter value: 10

Enter your choice: 1

Enter value: 20

Enter your choice: 3

Stack: 20 10

Enter your choice: 2

Popped element is 20

Enter your choice: 3

Stack: 10

Enter your choice: 4

Exit

2.Implement Queue using stack data structure.

CODE:

```
#include <stdio.h>
# define MAX 100

int stack1[MAX], stack2[MAX];
```

```
int top1 =-1, top2 = -1;

void push1(int x){
   if(top1 == MAX-1){
      printf("Stack 1 is full\n");
      return;
   }

   stack1[++top1] = x;
}

void push2(int x){
   if(top2 == MAX-1){
      printf("Stack 2 is full\n");
      return;
   }

   stack2[++top2] = x;
}

int pop1(){
   return stack1[top1--];
}
```

```
int pop2(){
   return stack2[top2--];
}

void enqueue(int x){
   push1(x);
}

void dequeue(){
   int x;

   if(top1 == -1 && top2 == -1){
      printf("Queue is empty\n");
      return;
   }

   if(top2 == -1){
      while(top1 != -1){
         x = pop1();
         push2(x);
      }
   }
```

```
    x = pop2();
    printf("Dequeued %d\n",x);

}

void display(){
    int i;

    if(top1 == -1 && top2 == -1){
        printf("Queue is empty\n");
        return;
    }

    printf("Queue: ");
    for(i = top2; i>=0; i--){
        printf("%d ",stack2[i]);
    }

    for (i = 0; i <= top1; i++) {
        printf("%d ", stack1[i]);
    }
```

```
    printf("\n");

}

int main(){
    int choice, value;

    printf("1.Enqueue\n2.Dequeue\n3.Display\n4.Exit\n");

    do{
        printf("Enter your choice: ");
        scanf("%d",&choice);

        switch(choice){
            case 1:
             printf("Enter value: ");
             scanf("%d",&value);
             enqueue(value);
             break;

            case 2:
             dequeue();
             break;
```

```
        case 3:
          display();
          break;

        case 4:
            printf("Exit\n");
            return 0;

      }
   }while(choice != 4);

   return 0;
}
```

OUTPUT:

```
1.Enqueue
2.Dequeue
3.Display
4.Exit
Enter your choice: 1
Enter value: 10
Enter your choice: 1
Enter value: 20
```

Enter your choice: 1

Enter value: 30

Enter your choice: 3

Queue: 10 20 30

Enter your choice: 2

Dequeued 10

Enter your choice: 2

Dequeued 20

Enter your choice: 3

Queue: 30

Enter your choice: 4

Exit

3. Write a C program using stack data structure to implement managing the history of webpage visited.

CODE:

```
#include <stdio.h>
#include <string.h>

#define MAX 100

char history[MAX][100];
int top = -1;
```

```
void visitPage(char page[]) {
    if (top == MAX - 1) {
        printf("History is full!\n");
        return;
    }

    top++;
    strcpy(history[top], page);

    printf("Visited: %s\n", page);
}

void goBack() {
    if (top == -1) {
        printf("No previous webpage available!\n");
        return;
    }

    printf("Going back from: %s\n", history[top]);
    top--;

    if (top >= 0)
```

```
        printf("Current webpage: %s\n", history[top]);
    else
        printf("No webpage in history.\n");
}

void displayHistory() {
    if (top == -1) {
        printf("History is empty!\n");
        return;
    }

    printf("\nWebpage History:\n");

    for (int i = top; i >= 0; i--) {
        printf("%s\n", history[i]);
    }
}

int main() {
    int choice;
    char page[100];

        printf("\nWebpage History\n");
```

```
printf("1. Visit Webpage\n");
printf("2. Go Back\n");
printf("3. Display History\n");
printf("4. Exit\n");

while (1) {
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
   case 1:
      printf("Enter webpage: ");
      scanf("%s", page);
      visitPage(page);
      break;

   case 2:
      goBack();
      break;

   case 3:
      displayHistory();
      break;
```

```
        case 4:
            return 0;

        default:
            printf("Invalid choice!\n");
      }
   }

   return 0;
}
```

OUTPUT:

```
Webpage History
1. Visit Webpage
2. Go Back
3. Display History
4. Exit
Enter your choice: 1
Enter webpage: https://www.youtube.com/
Visited: https://www.youtube.com/
Enter your choice: 1
Enter webpage:
https://www.bing.com/search?q=international+dogs+day+2026
```

Visited:
https://www.bing.com/search?q=international+dogs+day+2026

Enter your choice: 3

Webpage History:

https://www.bing.com/search?q=international+dogs+day+2026

https://www.youtube.com/

Enter your choice: 2

Going back from:
https://www.bing.com/search?q=international+dogs+day+2026

Current webpage: https://www.youtube.com/

Enter your choice: 4

4. Write a C program to sort a stack using temporary stack.

CODE:

```
#include <stdio.h>

#define MAX 100


int stack[MAX], temp[MAX];

int top = -1, tempTop = -1;


void push(int x){

   if (top == MAX - 1){
```

```
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = x;
}

int pop(){
    if (top == -1)
        return -1;

    return stack[top--];
}

void tempPush(int x){
    temp[++tempTop] = x;
}

int tempPop(){
    return temp[tempTop--];
}

void sortStack(){
    int x;
```

```
    while (top != -1){
        x = pop();

        while (tempTop != -1 && temp[tempTop] > x){
            push(tempPop());
        }

        tempPush(x);
    }

    while (tempTop != -1){
        push(tempPop());
    }
}

void display(){
    int i;
    printf("Sorted Stack:\n");

    for (i = top; i >= 0; i--){
        printf("%d ", stack[i]);
    }
```

```
    printf("\n");
}

int main(){
    int n, x, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (i = 0; i < n; i++){
        scanf("%d", &x);
        push(x);
    }

    sortStack();
    display();

    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
```

Enter elements:

12

78

34

23

56

Sorted Stack:

12 23 34 56 78

5. Given daily stock prices, find the stock span for each day.The span of a day is the maximum number of consecutive days, including the current day, for which the stock price is less than or equal to today's price.

Input:
[100, 80, 60, 70, 60, 75, 85]

Output:
[1, 1, 1, 2, 1, 4, 6]

CODE:

```
#include <stdio.h>

#define MAX 100


int stack[MAX];
```

```
int top = -1;

void push(int x){
   stack[++top] = x;
}

int pop(){
   return stack[top--];
}

int peek(){
   return stack[top];
}

int isEmpty(){
   return top == -1;
}

int main(){
   int price[MAX], span[MAX];
   int n, i;

   printf("Enter number of days: ");
```

```
scanf("%d", &n);

printf("Enter stock prices:\n");
for(i = 0; i < n; i++){
   scanf("%d", &price[i]);
}

for(i = 0; i < n; i++){
   while(!isEmpty() && price[peek()] <= price[i]){
      pop();
   }

   if(isEmpty())
      span[i] = i + 1;
   else
      span[i] = i - peek();

   push(i);
}

printf("Stock Span:\n");

for(i = 0; i < n; i++){
```

```
        printf("%d ", span[i]);
    }

    return 0;
}
```

OUTPUT:

```
Enter number of days: 7
Enter stock prices:
100
80
60
70
60
75
85
Stock Span:
1 1 1 2 1 4 6
```

6.Given an array, for each element find the nearest element to its that has a frequency higher than the current element. If no such element exists set value to -1. Implement using stack.

CODE:

```
#include <stdio.h>
#define MAX 100

int stack[MAX];
int top = -1;

void push(int x){
   stack[++top] = x;
}

int pop(){
   return stack[top--];
}

int peek(){
   return stack[top];
}

int isEmpty(){
   return top == -1;
}

int main(){
```

```
int arr[MAX], freq[MAX] = {0};
int result[MAX];
int n, i, j;

printf("Enter number of elements: ");
scanf("%d", &n);

printf("Enter array elements:\n");
for(i = 0; i < n; i++){
   scanf("%d", &arr[i]);
}


for(i = 0; i < n; i++){
   freq[arr[i]]++;
}

for(i = n - 1; i >= 0; i--){
   while(!isEmpty() && freq[arr[peek()]] <= freq[arr[i]]){
      pop();
   }

   if(isEmpty())
```

```
            result[i] = -1;
        else
            result[i] = arr[peek()];

        push(i);
    }

    printf("Result:\n");

    for(i = 0; i < n; i++){
        printf("%d ", result[i]);
    }

    return 0;
}
```

OUTPUT:

```
Enter number of elements: 6
Enter array elements:
2 1 1 3 2 1
Result:
1 -1 -1 2 1 -1
```

7.Given a pattern containing only I's and D's. I for increasing and D for decreasing. Device an algorithm using stack to print minimum number following that pattern. Digits from 1-9 and digits can repeat.

CODE:

```
#include <stdio.h>

#define MAX 100


int stack[MAX];

int top = -1;


void push(int x){

   stack[++top] = x;

}


int pop(){

   return stack[top--];

}


int main(){

   char pattern[MAX];

   int i, n;

   int num = 1;
```

```
    printf("Enter pattern (I/D): ");
    scanf("%s", pattern);

    n = 0;

    while(pattern[n] != '\0'){
       n++;
    }

    for(i = 0; i <= n; i++){
       push(num);
       num++;

       if(i == n || pattern[i] == 'I'){
          while(top != -1){
             printf("%d", pop());
          }
       }
    }

    return 0;
}
```

OUTPUT:

Enter pattern (I/D): DIDI

21435

8.Given an array of digits find the minimum possible sum of two numbers formed from digits of the array. All the digits of given array must be used to form the two numbers. Use Stack for implementation.

CODE:

```
#include <stdio.h>

#define MAX 100


int stack[MAX];

int top = -1;


void push(int x){

   stack[++top] = x;

}


int pop(){

   return stack[top--];

}
```

```
int main(){
   int a[MAX];
   int n, i, j, temp;
   int num1 = 0, num2 = 0;

   printf("Enter number of digits: ");
   scanf("%d", &n);

   printf("Enter digits:\n");
   for(i = 0; i < n; i++){
      scanf("%d", &a[i]);
   }

   for(i = 0; i < n - 1; i++){
      for(j = i + 1; j < n; j++){
         if(a[i] > a[j]){
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
         }
      }
   }
```

```
    for(i = n - 1; i >= 0; i--){
        push(a[i]);
    }

    for(i = 0; i < n; i++){
        if(i % 2 == 0)
            num1 = num1 * 10 + pop();
        else
            num2 = num2 * 10 + pop();
    }

    printf("Number 1 = %d\n", num1);
    printf("Number 2 = %d\n", num2);
    printf("Minimum Sum = %d\n", num1 + num2);

    return 0;
}
```

OUTPUT:

```
Enter number of digits: 6
Enter digits:
6 8 4 5 2 3
Number 1 = 246
Number 2 = 358
```

Minimum Sum = 604

9. Given an array and a positive integer, find the first negative integer for each window (contiguous subarray) of size k. If a window does not contain a negative integer, then print 0 for that window. Implement using Queue.

CODE:

```
#include <stdio.h>

#define MAX 100


int queue[MAX];

int front = 0;

int rear = -1;


void enqueue(int x){

   queue[++rear] = x;

}


int dequeue(){

   return queue[front++];

}


int peek(){
```

```
    return queue[front];
}

int isEmpty(){
    return front > rear;
}

int main(){
    int a[MAX];
    int n, k;
    int i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++){
        scanf("%d", &a[i]);
    }

    printf("Enter window size K: ");
    scanf("%d", &k);
```

```
front = 0;
rear = -1;

for(i = 0; i < k; i++){
   if(a[i] < 0)
      enqueue(i);
}

for(i = k; i <= n; i++){
   while(!isEmpty() && peek() < i - k){
      dequeue();
   }

   if(isEmpty())
      printf("0 ");
   else
      printf("%d ", a[peek()]);

   if(i < n && a[i] < 0){
      enqueue(i);
   }
}
```

```
    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter array elements:
-8 2 3 -6 1
Enter window size K: 2
-8 0 -6 -6
```

10.Given an m × n matrix: 0 → Empty cell, 1 → Fresh orange, 2 → Rotten orange. Every minute, a rotten orange makes its 4 adjacent fresh oranges rotten. Find the minimum time required for all fresh oranges to become rotten. If it is impossible, return -1.

CODE:

```
#include <stdio.h>
#define MAX 100

struct Cell{
    int row;
    int col;
};
```

```
struct Cell queue[MAX * MAX];

int front = 0;
int rear = -1;

void enqueue(int r, int c){
   queue[++rear].row = r;
   queue[rear].col = c;
}

struct Cell dequeue(){
   return queue[front++];
}

int isEmpty(){
   return front > rear;
}

int main(){
   int a[MAX][MAX];
   int m, n;
   int i, j;
   int fresh = 0;
```

```
int time = 0;

printf("Enter rows and columns: ");
scanf("%d %d", &m, &n);

printf("Enter matrix:\n");

for(i = 0; i < m; i++){
   for(j = 0; j < n; j++){
      scanf("%d", &a[i][j]);

      if(a[i][j] == 2){
         enqueue(i, j);
      }
      else if(a[i][j] == 1){
         fresh++;
      }
   }
}

int dr[] = {-1, 1, 0, 0};
int dc[] = {0, 0, -1, 1};
```

```
while(!isEmpty() && fresh > 0){
    int size = rear - front + 1;

    for(i = 0; i < size; i++){
        struct Cell current = dequeue();

        for(j = 0; j < 4; j++){
            int r = current.row + dr[j];
            int c = current.col + dc[j];

            if(r >= 0 && r < m && c >= 0 && c < n && a[r][c] == 1){
                a[r][c] = 2;

                fresh--;

                enqueue(r, c);
            }
        }
    }

    time++;
}
```

```
    if(fresh == 0)
        printf("Minimum time = %d\n", time);
    else
        printf("Minimum time = -1\n");

    return 0;
}
```

OUTPUT:

```
Enter rows and columns: 3 5
Enter matrix:
2 1 0 2 1 1 0 1 2 1 1 0 0 2 1
Minimum time = 2
```